Episodio 10 de 14: Sin Docker: el error escondido y la caché que se llena

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 9, «¿Es correcto? Detectores y la cota wait-free».
- **Episodio siguiente:** episodio 11, «El veredicto y los límites».

Lo esencial de este episodio:

- Con los mismos programas oficiales, Docker no cambia nada: de 270 puntos, 259 cambian menos de un 10 %.
- El programa del equipo, bench.cpp, tenía compartición falsa: sus 10 pilas cabían en 2 líneas de caché. Al alinearlas, Autores pasa de 35 a 120 Mop/s.
- Con un millón de pilas, alinear frena entre un 24 % y un 27 %, porque las pilas dejan de caber en la caché L3: cada operación saca de la RAM entre 207 y 265 bytes más.

6.6 La versión sin Docker

Qué se hizo. Se midieron tres tandas directamente sobre Ubuntu, sin Docker. Ninguna medición falló.

Tanda	Programa	Algoritmos	Mediciones	A 90 °C o más
`oficial` y `extra`	Los binarios oficiales, compilados fuera de Docker	Los 5 del paper	4 660, más la memoria retenida y la receta exacta	17, con un máximo de 98 °C
`equipo`	`codigo/bench.cpp`	Autores, Libstdc++, Williams y Mutex	2 976	8, con un máximo de 99 °C
`alineado`	Una copia de `bench.cpp` con `alignas(128)`	Los mismos 4	2 160	1, con un máximo de 93 °C

Las de 90 °C o más están repartidas entre escenarios y no se descartaron. Las pruebas de corrección de la Parte 6.5 ya se hicieron sin Docker desde el principio.

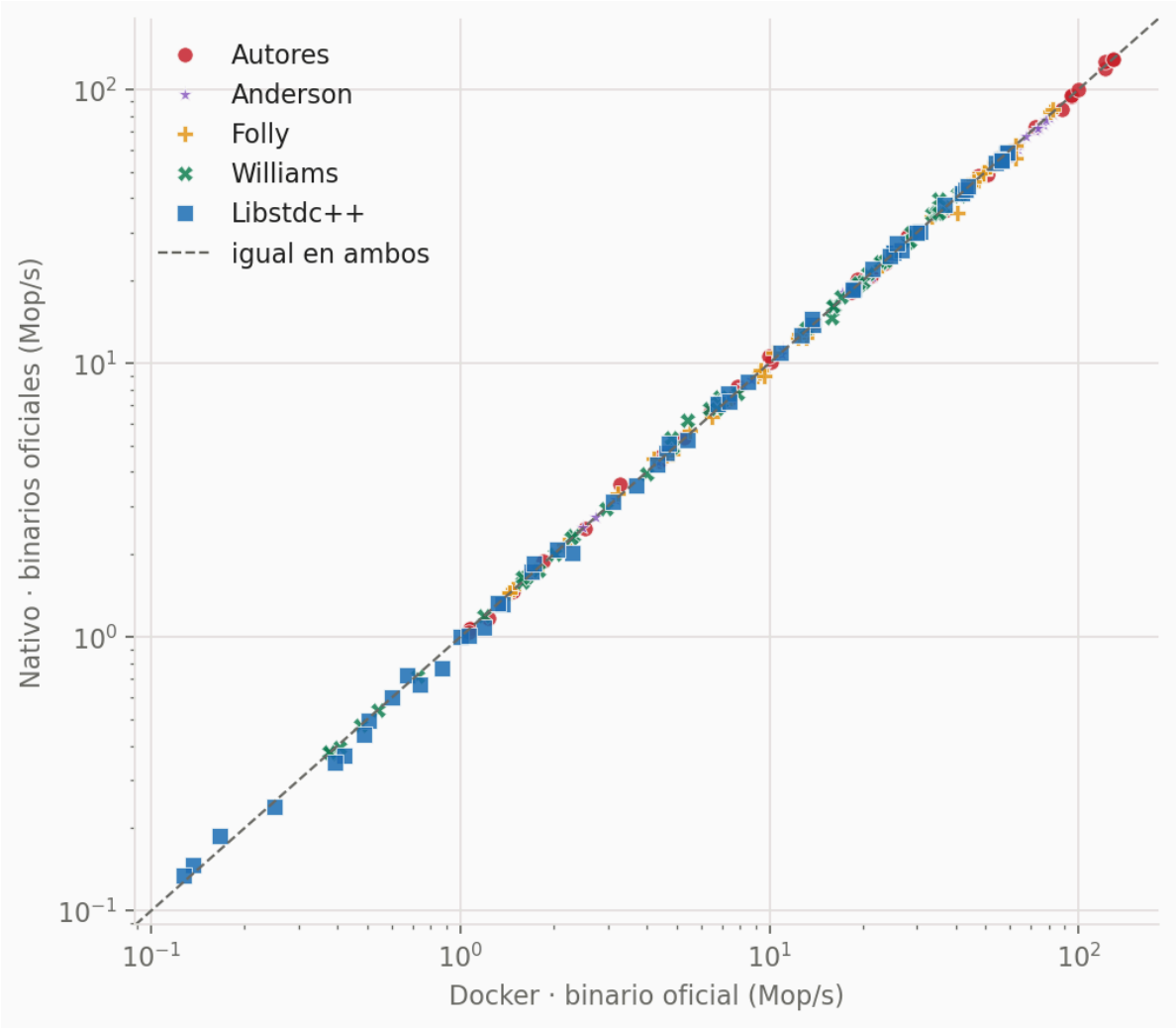
Se empezó por `bench.cpp` porque al principio los programas oficiales no compilaban fuera del contenedor (Parte 8). Cuando se arregló, la cola `pendientes.sh` midió los oficiales y la versión alineada de un tirón, la noche del 6 al 7 de noviembre de 2026. Aquí van en el orden más útil para entenderlos: primero la comparación limpia.

Con los programas oficiales: Docker no cambia nada

Aquí el programa es el mismo dentro y fuera. Pero ojo, tampoco cambia solo Docker. Fuera del contenedor cambian también estas piezas:

	Con Docker	Sin Docker
g++-12	12.3.0	12.4.0
glibc	2.35	2.39
Boost	1.74	1.83
jemalloc	5.2.1	5.3.0
Compilación de Folly	`getdeps.py`	CMake directo

Así que, si algo cambia, no se puede culpar a Docker sin más. Y si nada cambia, es que todas esas piezas juntas tampoco importan aquí.



Cada punto de la figura 5 con los programas oficiales, con Docker y sin Docker

Cómo leerla. Cada punto es un algoritmo, en un escenario y con un número de hilos. Si dentro y fuera da lo mismo, el punto cae sobre la línea discontinua. Y caen todos encima.

Algoritmo (sin Docker ÷ con Docker)	Mediana	Mínimo	Máximo
Autores	1,00×	0,95×	1,11×
Anderson	0,99×	0,95×	1,06×
Folly	1,00×	0,87×	1,06×
Williams	1,01×	0,92×	1,15×
Libstdc++	1,00×	0,87×	1,12×

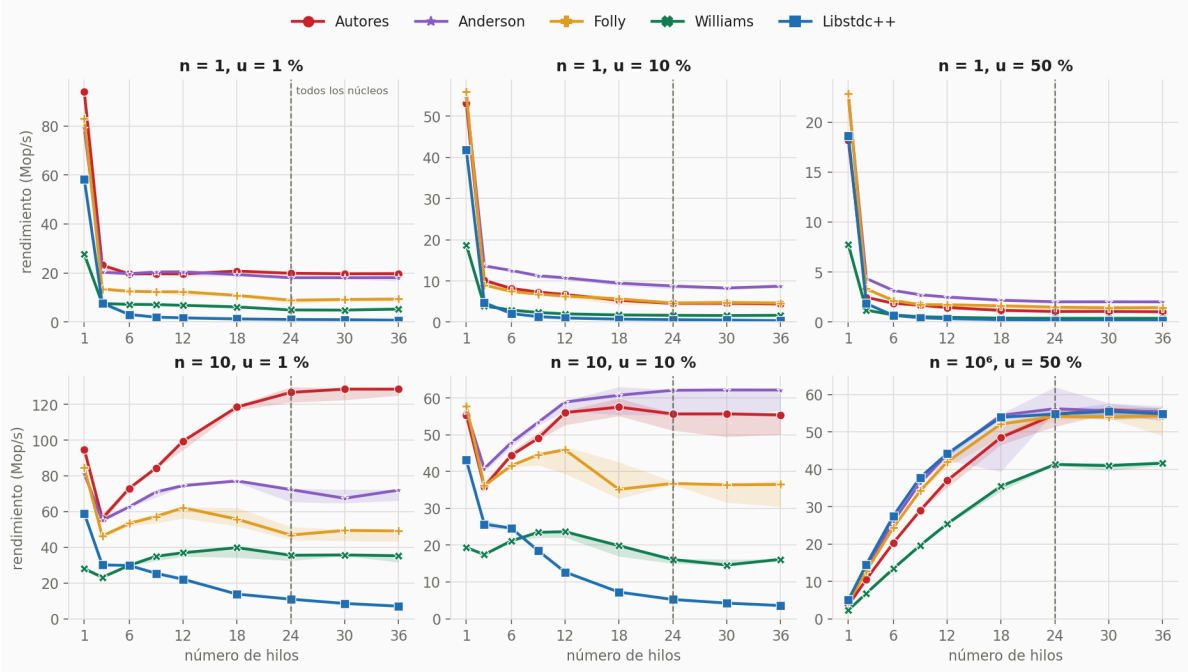
De los 270 puntos, 236 cambian menos de un 5 % y 259 menos de un 10 %. Los que más se mueven son puntos con contención (n = 1 o n = 10), sobre todo de Libstdc++ y Williams, que es justo donde las rondas más varían.

Conclusión: Docker no cambia el rendimiento de los programas oficiales. Antes era una deducción a partir de `bench.cpp`; ahora está medido. No se puede descartar del todo que dos efectos contrarios se anulen, pero tendría que pasar igual en los 270 puntos.

Lo demás también sale casi calcado: la memoria retenida (Parte 6.3), el efecto de separar los hilos (Parte 6.4) y la latencia sin contención (Parte 6.2). Hay dos excepciones, y ninguna permite culpar a Docker:

- **Williams entre clústeres.** Con dos hilos en clústeres distintos da 0,09 Mop/s con Docker y 0,70 sin Docker (Parte 6.4). Es la diferencia más grande y más clara. Hay otro punto raro, Folly con 8 hilos compactos, pero ese es inestable también con Docker (ver la misma Parte 6.4). Williams usa `boost::atomic`, y Boost es una de las piezas que cambian. Es una sospecha, no una medida.
- **La latencia con contención.** Algunas medianas cambian mucho; por ejemplo, el `load` de Williams con p = 24 pasa de 3 104 a 6 106 ns. Pero una misma medida cambia todavía más de una ronda a otra (Parte 6.2), así que estas diferencias no sirven para saber si Docker influye.

La figura 5 sin Docker, para compararla con la de la Parte 6.1:



Rendimiento sin Docker con los programas oficiales

Con `bench.cpp` del equipo: cambian dos cosas a la vez

Lo primero que hay que saber. Esto no es «lo mismo, pero sin Docker»: cambia también el programa. `bench.cpp` cubre Autores, Libstdc++, Williams y Mutex. **Folly y Anderson no están.**

Es como cambiar de cronómetro y de pista a la vez: si sale otro tiempo, no sabes a cuál de los dos culpar. Esto es todo lo que cambia:

	Con Docker	Sin Docker
Programa	`bench_stack`, el oficial	`codigo/bench.cpp`, del equipo
Pilas en memoria	Cada una en su sitio (`alignas(128)`)	Seguidas, sin alinear
Nodos por pila	20 + T/n	16, o 2 si n > 10
Generador de azar	Uno rápido	`mt19937_64`, más lento
Compilación	g++ 12.3, `-O3`	g++ 12.4, `-O2`
Williams	Con `boost::atomic`	Con `std::atomic`
Autores	Versión del artefacto (la cuenta sube)	Versión publicada (la cuenta baja)
Biblioteca C y jemalloc	glibc 2.35, jemalloc 5.2.1	glibc 2.39, jemalloc 5.3.0

A `bench.cpp` se le añadió jemalloc con `LD_PRELOAD` y se le pidió n = 10^6, igual que al oficial, sin tocar su código. Para separar efectos se hicieron dos controles.

Control 1: ¿Docker cambia algo por sí solo? No

Se compiló **el mismo código** dentro y fuera del contenedor, y se midió alternando: un punto fuera, el mismo dentro, con el orden al azar. Fueron 48 puntos y 3 rondas.

Resultado (dentro ÷ fuera)	Mediana	Mínimo	Máximo
Los 48 puntos	**1,02×**	0,91×	1,13×

Docker no es una máquina virtual. El programa corre sobre el mismo núcleo de Linux y en la misma CPU; solo tiene una valla alrededor. Es como correr en la misma pista con un dorsal puesto: el dorsal no te frena. La diferencia máxima (dentro va hasta un 13 % **más rápido**, con 1 hilo) tampoco hay por qué achacarla a Docker: dentro hay otra versión del compilador, de la biblioteca C y de jemalloc.

Conclusión: las diferencias entre las dos tandas vienen del programa, no de Docker. Los programas oficiales lo confirmaron después, como se ha visto arriba.

Control 2: por qué n = 10 sale 3 veces más lento

Este fue el hallazgo más claro. Mira la tabla con 24 hilos, en Mop/s:

Escenario	Autores: Docker → sin Docker	Williams	Libstdc++
n = 1, u = 1 %	19,8 → 21,0	4,8 → 4,0	1,2 → 0,9
n = 1, u = 10 %	4,4 → 5,0	1,6 → 0,9	0,6 → 0,4
n = 1, u = 50 %	1,1 → 1,2	0,4 → 0,2	0,2 → 0,1
n = 10, u = 1 %	**121,9 → 35,4**	34,7 → 11,4	10,8 → 6,7
n = 10, u = 10 %	**55,4 → 18,4**	16,1 → 5,8	5,4 → 3,6
n = 10^6, u = 50 %	55,4 → 78,8	40,4 → 50,7	56,3 → 82,5

Con n = 1, Autores coincide. Con n = 10 rinde **entre 3 y 3,4 veces menos**. Algo pasa solo cuando hay varias pilas.

La historia. Imagina 10 personas, cada una con su propio cuaderno. Nadie comparte nada. Pero resulta que los 10 «cuadernos» son 10 renglones de **la misma hoja**, y esa hoja solo puede estar en una mano. Cada uno escribe solo en su renglón, y aun así la hoja viaja de mano en mano como si todos compartieran el mismo. Eso es la **compartición falsa** (*false sharing*): no se comparte ningún dato, pero se comparte el papel.

Lo mismo, con números. Los datos viajan en líneas de caché de 64 bytes. Esto ocupa cada pila en `bench.cpp`:

Implementación	Bytes por pila	Las 10 pilas ocupan
Autores	8	80 bytes: **unas 2 líneas**
Libstdc++	16	160 bytes: unas 3 líneas
Williams	16	160 bytes: unas 3 líneas
Mutex	56	560 bytes: unas 9 líneas

Es decir, con Autores las «10 pilas» se comportan casi como 2. Su propio diseño compacto, 8 bytes frente a 16, juega aquí en su contra.

Lo mismo, en código. El programa oficial pide espacio de sobra para cada pila. `bench.cpp`, no:

```
class alignas(128) atomic_stack { ... };          // programa oficial: cada pila en su sitio
template <class I> struct Pila { ... };           // bench.cpp: las pilas quedan seguidas
```

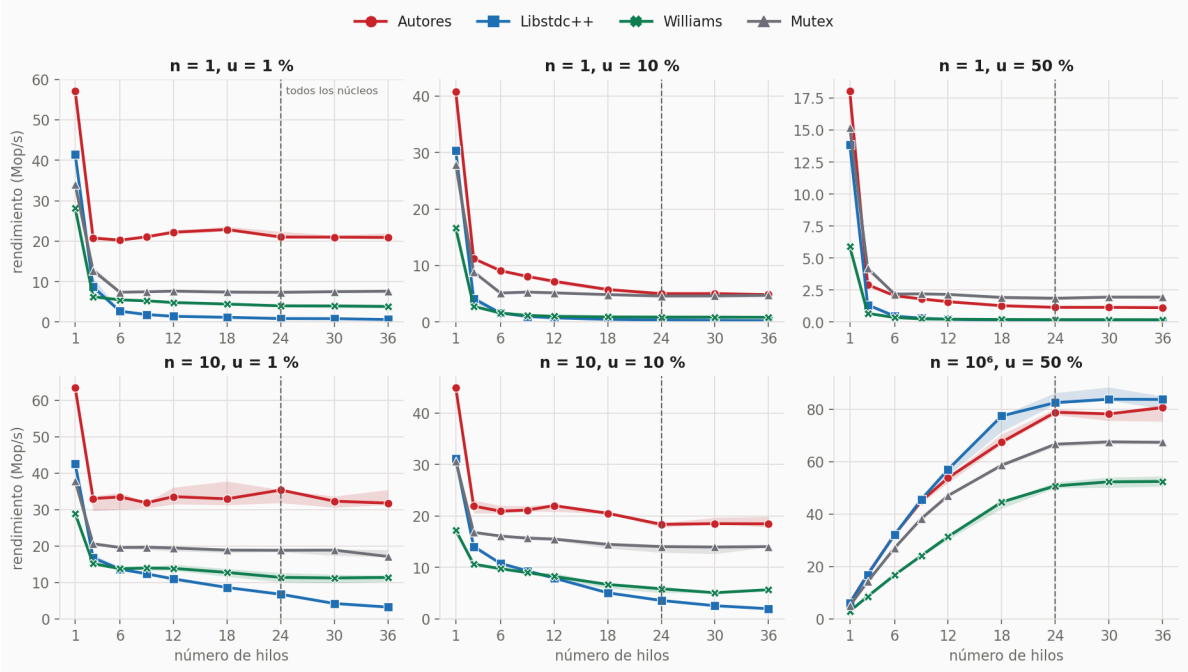
La prueba. Una hipótesis no vale hasta que se comprueba. En una copia de `bench.cpp` se añadió solo una palabra, `alignas(128)`, y se midieron las dos versiones alternando. El archivo del equipo no se tocó.

24 hilos, n = 10, u = 1 %	Sin alinear	Alineado	Mejora	Oficial, en Docker
Autores	32,5	**118,6**	**3,65×**	121,9
Williams	10,6	26,2	2,47×	34,7
Mutex	17,7	23,3	1,32×	—
Libstdc++	6,8	8,2	1,21×	10,8

Y el control que hace creíble todo lo anterior: con n = 1 solo hay una pila, sin vecinas que estorbar, y la palabra añadida no cambia nada (0,95× a 1,05×). Si hubiera mejorado también ahí, la explicación sería falsa.

Autores gana más porque tiene más pilas por línea y porque nada más lo frena. Libstdc++ gana menos con 24 hilos porque ya lo frena su propio candado; con 6 hilos, en cambio, mejora 2,04×.

Consecuencia práctica: en la gráfica siguiente, los paneles n = 10 están limitados por el programa de pruebas, no por el algoritmo. No hay que sacar de ahí conclusiones sobre los algoritmos.

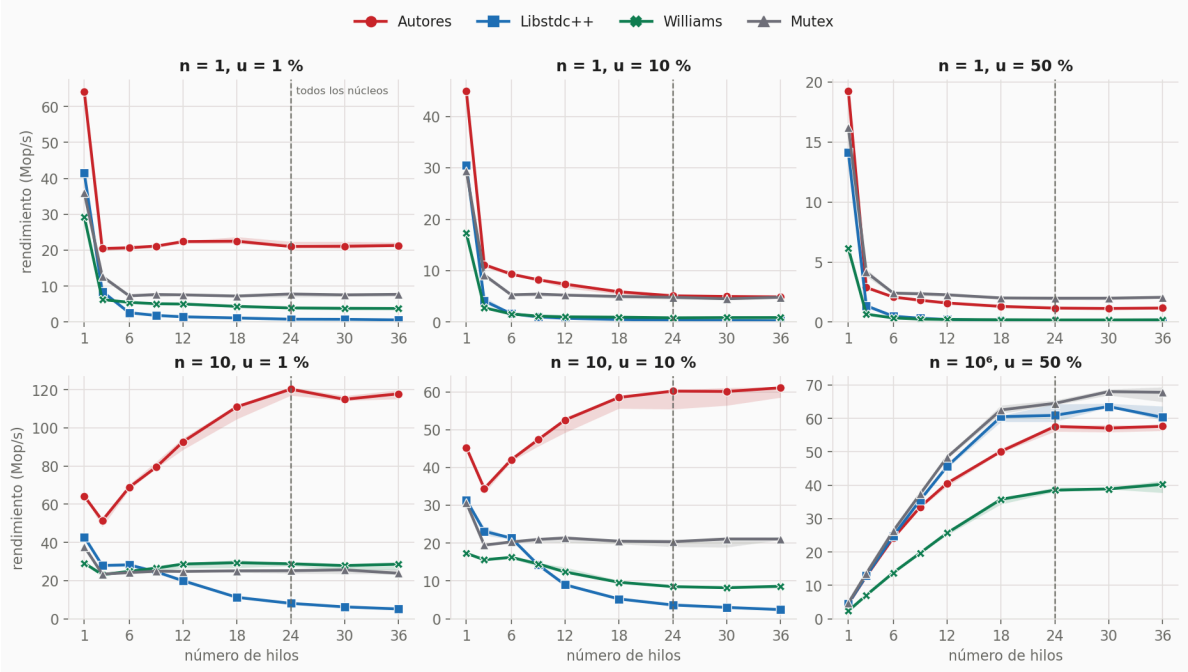


Rendimiento sin Docker con bench.cpp del equipo

La batería completa, con las pilas alineadas. El control anterior solo miraba unos pocos puntos. Después se repitieron enteras la figura 5 y la afinidad con la copia alineada: 2 160 mediciones, ninguna fallida. Esto sale con 24 hilos (tabla «Batería sin Docker con las pilas alineadas» de `tablas.md`):

24 hilos: alineado ÷ sin alinear	Autores	Williams	Libstdc++	Mutex
n = 1, los tres escenarios	1,00× a 1,02×	0,93× a 0,98×	0,92× a 1,01×	1,04× a 1,07×
n = 10, u = 1 %	**3,40×** (35,4 → 120,3)	2,54×	1,20×	1,34×
n = 10, u = 10 %	**3,28×** (18,4 → 60,2)	1,46×	1,02×	1,45×
n = 10^6, u = 50 %	**0,73×** (78,8 → 57,6)	0,76×	0,74×	0,97×

Con n = 10 se confirma lo que decía el control: Autores sube a 120 Mop/s, cerca de los 127 del programa oficial sin Docker. Y el control de n = 1 también se cumple: ahí la palabra añadida apenas cambia nada.



Rendimiento sin Docker con bench.cpp y las pilas alineadas

Compárala con la anterior: en los paneles n = 10, Autores ya no va a la par de los demás, sino muy por encima, como en el programa oficial.

Y una sorpresa con n = 10^6: alinear frena. Autores, Williams y Libstdc++ pierden entre un 24 % y un 27 %; el Mutex, solo un 3 %. No se esperaba. La sospecha era el tamaño. Sin alinear, cada pila ocupa 8 bytes en Autores, 16 en Libstdc++ y Williams y 56 en el Mutex (medido con `sizeof`). Con `alignas(128)`, todas ocupan 128. Un millón de pilas pasan así de 8-16 MB, que caben en los 36 MB de caché L3 de este PC, a 128 MB, que no caben. El Mutex ya ocupaba 56 MB sin alinear: no cabía ni antes, y por eso casi no cambia.

Piensa en un fichero con un millón de fichas. Si cada ficha es una tarjeta pequeña, el fichero entero cabe en el cajón de tu mesa (la L3) y casi nunca tienes que levantarte. Si metes cada ficha en una carpeta grande, ya no cabe, y muchas consultas te obligan a ir al almacén (la RAM). Con cien mil fichas, en cambio, las carpetas todavía caben en el cajón.

Eso da dos predicciones que se pueden comprobar:

1. Con n = 10^6, la versión alineada debe sacar **más bytes de la RAM por cada operación**.
2. Con n = 10^5, las pilas alineadas ocupan 12,8 MB y todavía caben en la L3, así que **alinear no debe frenar**.

Se comprobaron el 7 de noviembre de 2026 con `herramientas/alinear_memoria.py`. Usa los mismos contadores de la controladora de memoria que la Parte 6.1, con 24 hilos, u = 50 % y 3 rondas. Esta vez las dos versiones de cada punto se midieron **seguidas y en orden al azar**, así que las compara el mismo estado de la máquina (tabla «Por qué alinear frena con n = 10^6» de `tablas.md`):

24 hilos, u = 50 %	Autores	Libstdc++	Williams	Mutex
n = 10^6: alineado ÷ sin alinear, en	0,72× a 0,74×	0,73× a 0,75×	0,74× a 0,76×	0,95× a 0,97×

24 hilos, u = 50 %	Autores	Libstdc++	Williams	Mutex
cada ronda				
n = 10^6: bytes de RAM por operación	480 → **687**	512 → **756**	643 → **908**	673 → 701
n = 10^5: alineado ÷ sin alinear, en cada ronda	1,00×	0,94× a 1,02×	0,95× a 0,97×	1,01× a 1,02×

Las dos predicciones se cumplen:

- **Con n = 10^6**, alinear añade entre 207 y 265 bytes de RAM a cada operación (un 41-48 % más), y el rendimiento baja casi en la misma proporción. En el Mutex solo añade 28 bytes (un 4 %) y apenas pierde. Al alternar salen los mismos cocientes que en la batería, que se midió con horas de diferencia (0,73×-0,76× y 0,97×), así que la diferencia no viene de que la máquina fuera más lenta esa noche.
- **Con n = 10^5**, alinear casi no cambia nada: de 0,94× a 1,02×. Solo Williams pierde algo, un 3-5 %.

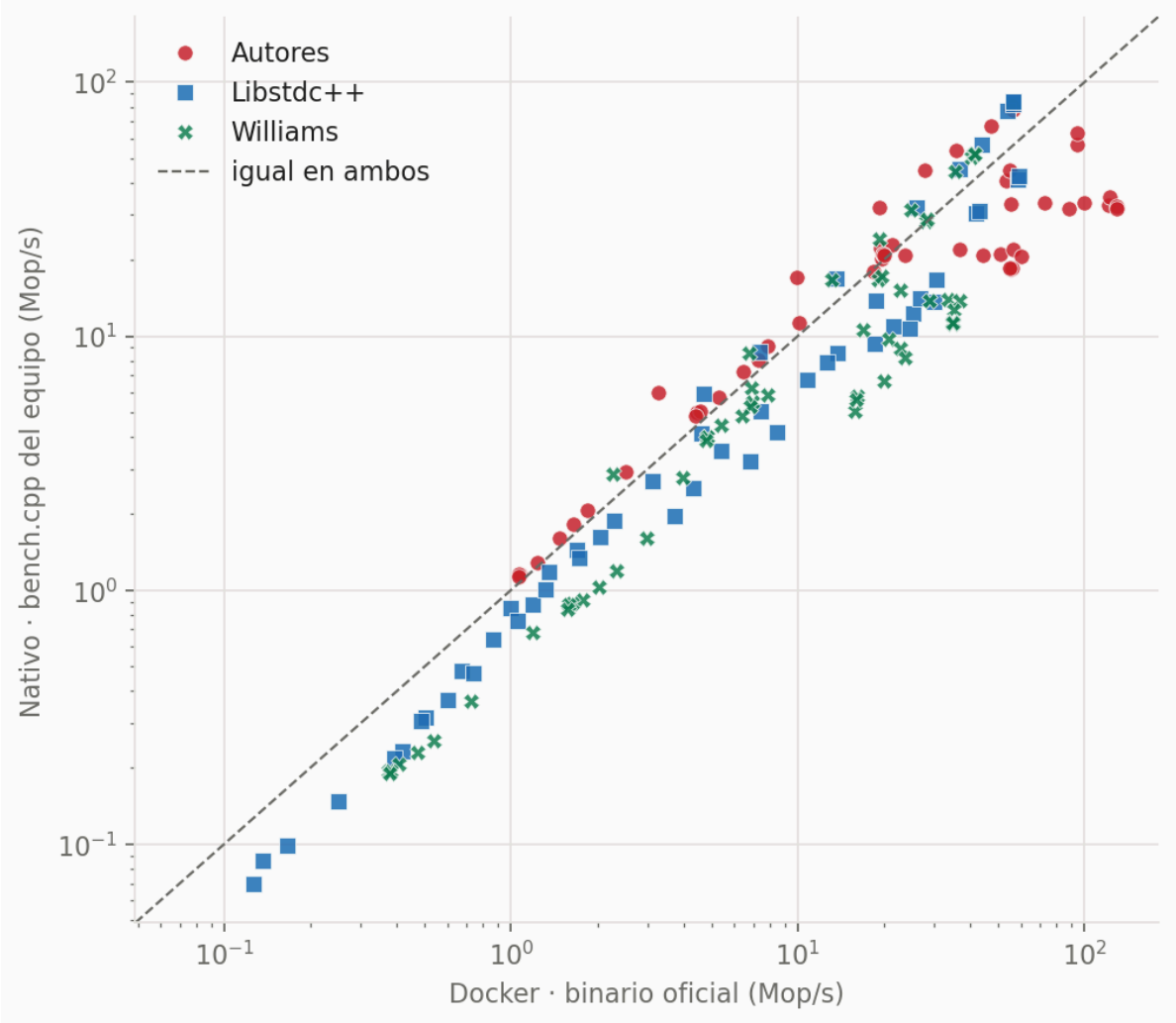
Hay un detalle más. Con n = 10^6, todas las versiones mueven entre 31 y 36 GB/s, muy cerca del techo de la RAM con accesos al azar (unos 37 GB/s, Parte 6.1). Por eso, si cada operación necesita más bytes, caben menos operaciones por segundo. Esto refuerza que la RAM es el freno con n = 10^6, aunque la salvedad de la Parte 6.1 sigue en pie: con un solo módulo de RAM no se puede subir el techo para comprobarlo del todo.

Ojo con las cifras absolutas: aquí Autores sin alinear da 65,1 Mop/s, y en la batería, 78,8. Esta prueba dura 10 s por punto y la batería, 2 s. No se investigó si es por eso. Por eso solo se comparan cocientes, que sí coinciden.

Esto corrige una deducción anterior. Se pensaba que `bench.cpp` iba más rápido con n = 10^6 porque sus pilas tienen 2 nodos en vez de 20. Pero la copia alineada sigue teniendo 2 nodos y aun así rinde como el programa oficial: 57,6 frente a 54,7 Mop/s en Autores. Ya está medido que, dentro de `bench.cpp`, tener las pilas apretadas ahorra unos 200-260 bytes de RAM por operación. Que eso explique toda la ventaja sobre el programa oficial sigue siendo una deducción, porque no se midió cuánto ocupa cada pila en el programa oficial.

Las otras diferencias

La gráfica siguiente pone cada punto en su sitio. Si las dos tandas coincidieran, todos caerían sobre la línea discontinua. Los puntos rojos de abajo a la derecha son Autores con n = 10, la compartición falsa que acabamos de ver.



Cada punto de la figura 5, con Docker y sin Docker

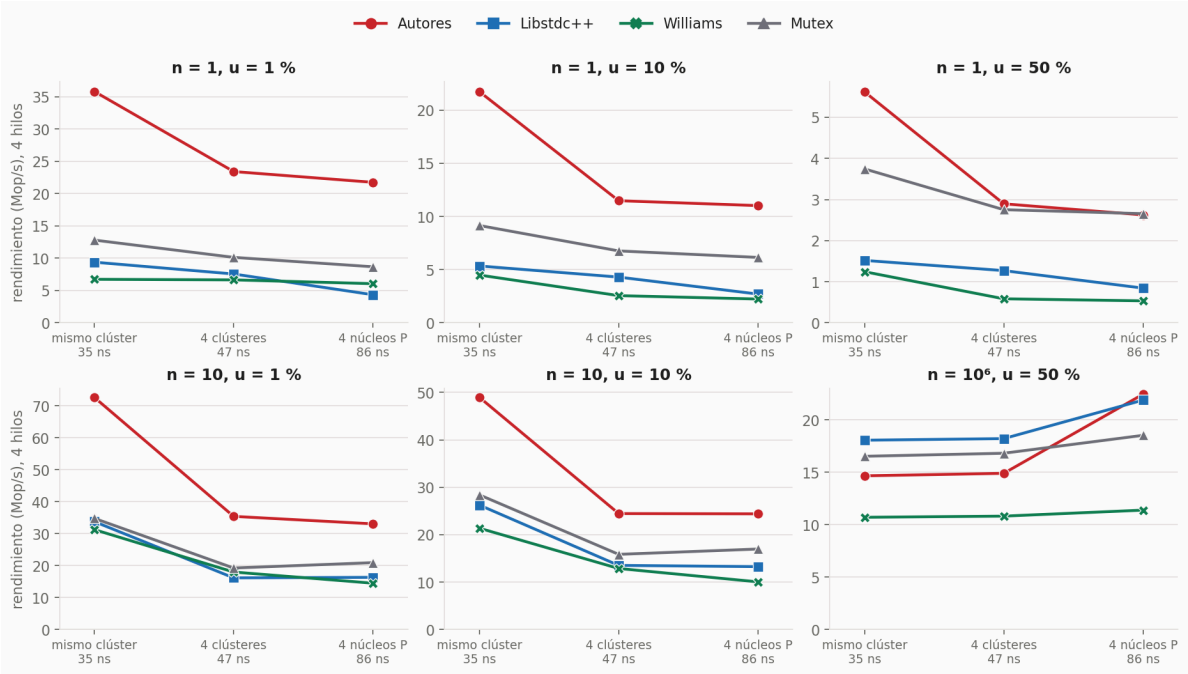
- **Williams y Libstdc++ rinden menos con `bench.cpp`,** incluso con n = 1, donde no hay compartición falsa. Mediana de los 54 puntos: 0,55× y 0,71×. Comparando con los programas oficiales también sin Docker, sale exactamente lo mismo (0,55× y 0,71×, calculado de los dos CSV), así que la brecha viene del programa, no del entorno. En Williams, usar `boost::atomic` explica una parte: da entre un 12 % y un 25 % más con 24 hilos, pero no cierra la brecha. El resto no se investigó. Hay candidatos, como `-O3` frente a `-O2` o el generador de azar, pero sin probar.
- **Con n = 10^6 y 24 hilos, `bench.cpp` va un 26-46 % más rápido.** Antes se atribuía a que sus pilas tienen 2 nodos en vez de 20. La batería alineada apunta más bien a que sus pilas están apretadas en memoria (ver «Y una sorpresa con n = 10^6», arriba). Dentro de `bench.cpp` eso ya está medido con los contadores de la RAM. La comparación con el programa oficial sigue siendo una deducción.

Lo que sí coincide

Esto es lo más importante, porque son dos programas escritos por personas distintas:

- **Quién gana.** Entre los 3 algoritmos comunes, coincide en 49 de 54 puntos. De las 5 diferencias, 4 son Autores contra Libstdc++ con n = 10^6, y hay márgenes muy pequeños.
- **Autores gana a Williams** en los 6 escenarios (0,17× a 0,64×), **y a Libstdc++** con contención (0,04× a 0,19×), con la compartición falsa en contra y todo.

- **La distancia.** Con 4 hilos, separar le cuesta a Autores entre un 35 % y un 51 %, frente al 32-54 % en Docker. Con n = 10^6 cambia un 2 % o menos en ambos. También aquí el efecto se apaga con 8 hilos o más.



Rendimiento con 4 hilos, de más cerca a más lejos, sin Docker

Se ve el mismo escalón de la Parte 6.4: una caída grande al salir del clúster, y luego casi plano.

Una excepción. El +25 % de Williams al separar 4 hilos con n = 1 y u = 1 % no aparece aquí (≈ −1 %). Era una rareza, y solo la tiene el programa oficial. Ahora está comprobado: el programa oficial sin Docker la repite (+26 %), y `bench.cpp`, no.

La sorpresa del Mutex

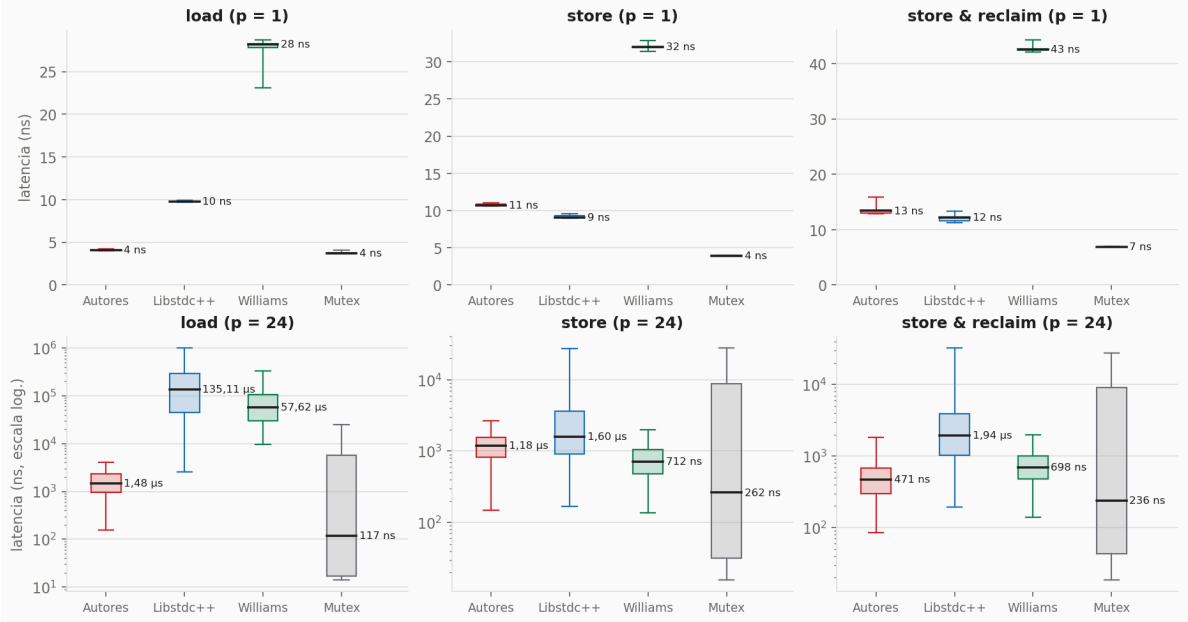
El candado más simple, que no está en el paper, **gana a Autores con n = 1 y u = 50 %**, desde 3 hilos hasta 36. Con 24 hilos: 1,86 frente a 1,15 Mop/s. Con las pilas alineadas, igual: 1,99 frente a 1,17.

¿Cómo puede ganar algo tan simple? Recuerda: en el Mutex, quien espera **se duerme**. Si hay 24 hilos pero 22 duermen, el cuaderno casi no viaja. En Autores, todos siguen despiertos y sus `CAS` fallan una y otra vez. Es una explicación razonable, pero no se comprobó.

Esto no le quita valor al paper. **Wait-free no significa «el más rápido»; significa «nadie se queda atascado».** Si el hilo que tiene el candado se detiene, en el Mutex se paran todos. En Autores, no.

Latencia sin Docker con `bench.cpp`

La latencia con los programas oficiales sin Docker está en la Parte 6.2. Esta es la de `bench.cpp`, que es otra prueba:



Latencia sin Docker con bench.cpp del equipo

Medianas, en nanosegundos:

Operación	Autores	Williams	Libstdc++	Mutex
`load`, p = 1	4	28	10	4
`load`, p = 24	1 478	57 621	135 108	117
`load`, p = 56	1 400	13 561	5 751	205
`store`, p = 1	11	32	9	4
`store`, p = 24	1 184	712	1 599	262
`store`, p = 56	1 206	2 367	1 540	253
`store & reclaim`, p = 1	13	43	12	7
`store & reclaim`, p = 24	471	698	1 937	236
`store & reclaim`, p = 56	1 353	4 379	1 972	228

Cuatro avisos para leerla bien:

1. **Estos números no se comparan con los de Docker.** Es otra prueba. `bench.cpp` descuenta lo que tarda el propio reloj y, con p = 1, promedia 16 operaciones: por eso Autores da 4 ns y no 14. Con contención, sus escritores eligen al azar entre 64 objetos compartidos; en el programa oficial, cada uno usa siempre el suyo.
2. **Con p = 1, el orden sí coincide.** En `load`: Autores, Libstdc++, Williams. En `store` y `store & reclaim`: Libstdc++, Autores, Williams.
3. **La mediana del Mutex engaña.** Parece el mejor (117 ns). Pero mira su caja: una de cada cuatro operaciones pasa de 5,8 µs y el p99 llega a 25 µs, frente a 4 µs de Autores. Es como un banco donde casi siempre te atienden al instante, porque la mayoría de los clientes se ha quedado dormida en la sala de espera. La persona típica va rápido, pero la que espera, espera mucho. Por eso la Parte 6.2 mira también las colas.
4. **Con contención, las medianas bailan.** El `load` de Libstdc++ con p = 56 va de 2,5 a 40,7 µs según la ronda. Que tarde 135 µs con p = 24 y 5,8 µs con p = 56 **no** significa que mejore con más hilos: su mediana baja, pero su p99 sube de 1 a 6,7 ms. Con más hilos que núcleos, la mayoría ni siquiera está corriendo.

¿Cambia el veredicto?

Con los programas oficiales ya se pueden contrastar las 15 filas de la Parte 7, también las que dependen de Folly y de Anderson. La columna de `bench.cpp` es la que se tenía antes, sin esos dos:

Filas	Sin Docker, programas oficiales	Sin Docker, `bench.cpp`
1 y 2 (gana a Libstdc++ y a Williams)	**Igual que con Docker.** Libstdc++ rinde 0,05× a 0,17× con contención y empata con n = 10^6; Williams, 0,25× a 0,76× en los 6 escenarios	Se sostienen
3 (Folly)	**Igual: no se reproduce.** Gana 3, empata 2 y pierde 1	No se podía comprobar
4 (Anderson con u pequeño)	**Igual: se reproduce.** Autores gana un 10 % con n = 1 y un 75 % con n = 10	No se podía comprobar
5 (Anderson con n grande)	**Igual: no se reproduce.** Autores pierde con pocos hilos y empata desde 24	No se podía comprobar
6 (Anderson con mucha contención)	**Igual: se reproduce.** Anderson rinde 1,93× y 1,90×	No se podía comprobar
7 (`load`)	**Se sostiene, por poco**: con contención, los rangos se solapan con Anderson y Folly	Se sostenía
8 y 9 (`store`)	**Igual que con Docker**: la 8 no se reproduce y la 9 es un empate con Anderson	No se podían comprobar del todo; con p = 24, Williams ganaba a Autores
10 (`store & reclaim` con p = 1)	**Igual: se reproduce** en los cinco	Se sostenía
11 (`store & reclaim` con contención)	**Ya no se confirma**: la mejora se queda en un 1-4 %, dentro del ruido	Se sostenía
12 (latencia de Williams)	**Igual: en parte**, algo más débil	Con contención tampoco era siempre el peor
13 (memoria)	**Igual: se reproduce.** Anderson, 20 256 bloques con 24 hilos	No se podía comprobar
14 y 15	Se probaron sin Docker desde el principio	Igual

Los datos de cada fila están en las Partes 6.1 a 6.4 y en las tablas «(nativo, binarios oficiales)» y «(nativo_oficial)» de `tablas.md`.